

# Cheerful Bot

## Emotion-Aware Mental Health Chatbot

Technical Project Report — May 2026

Mostafa Fouad : 2405647

Ahmad Osama : 2405610

Abdallah ibrahim : 2405628

|               |                                                    |
|---------------|----------------------------------------------------|
| Project Title | Cheerful Bot — Emotion-Aware Mental Health Chatbot |
| Version       | v2.0 — Kaggle Dataset Edition                      |
| Date          | May 2026                                           |
| Technology    | Python 3 · Scikit-learn · Streamlit · SQLite       |
| Languages     | English · Arabic · French                          |

## 1. Objective

---

Cheerful Bot is a locally-running, privacy-first mental health chatbot designed to detect the emotional tone of user messages and respond with contextually appropriate, uplifting therapeutic replies. The core goal is to lower the barrier to emotional support by providing an always-available, judgment-free conversational interface — without relying on any cloud APIs or external services.

**The system targets three complementary sub-goals:**

- Accurate emotion detection across 11 emotion classes from free-text input.
- Multilingual accessibility so users can interact in English, Arabic, or French.
- Therapeutic response quality — cheerful, supportive replies tailored to the detected emotion.

## 2. Dataset Description

---

The model is trained on the Kaggle Emotion Detection from Text dataset, a collection of short English-language tweets annotated with one of 12 emotion labels. The dataset was selected for its breadth of emotional expression in informal, everyday language.

| Property       | Detail                                                                                             |
|----------------|----------------------------------------------------------------------------------------------------|
| Source         | Kaggle — Emotion Detection from Text                                                               |
| Total Samples  | 39,653 tweets                                                                                      |
| Emotion Labels | anger, empty, enthusiasm, fun, happiness, hate, love, neutral, relief, sadness, surprise (11 used) |
| Format         | CSV with columns: text, emotion                                                                    |
| Quality        | No missing values; no duplicate records                                                            |
| Language       | English (multilingual support added at inference via translation)                                  |
| Split          | 70% train / 15% validation / 15% test                                                              |

*Note on class imbalance: the raw Kaggle dataset has a significant class imbalance (up to 77.8:1 across some emotion pairs). The final trained model applies balanced class weighting in the Logistic Regression classifier to counteract this.*

## 3. Methodology

---

The system follows a classical NLP pipeline enhanced with a custom negation-tagging pre-processing step and a Case-Based Reasoning (CBR) retrieval layer. The end-to-end pipeline is illustrated below.

### 3.1 Text Pre-Processing (nlp\_cleaner.py)

Every user message passes through the following pipeline — identically at both training time and inference time to prevent train/inference mismatch:

- Contraction expansion: contractions such as “can’t” are expanded to “can not” so that negation tokens survive.
- Lowercasing and non-alpha removal: only [a-z], underscores, and spaces are retained.
- Negation tagging (key innovation): negation triggers (not, never, no, nor, ...) are fused with the next meaningful token. For example, “not happy” becomes the single token NOT\_happy, and “can’t feel” becomes NOT\_feel.
- Safe stop-word removal: a curated stop-word list that excludes emotionally critical words (very, bad, alone, empty, ...) and all negation triggers.
- Rule-based lemmatisation: suffix-stripping (e.g. “happily” → “happi”). NOT\_\* tokens are intentionally skipped to preserve their meaning.
- Short token removal: tokens of one character or fewer are dropped.

### 3.2 TF-IDF Vectorisation

Cleaned text is transformed into a numerical feature vector using TF-IDF with up to 20,000 features and 1–3-gram ranges (unigrams, bigrams, and trigrams). Trigrams are essential for capturing patterns such as “not very happy” as a distinct phrase. Sublinear TF scaling ( $\log(1+tf)$ ) suppresses the influence of highly repeated terms.

### 3.3 Emotion Classification (Logistic Regression)

A Logistic Regression classifier (one-vs-rest,  $C = 1.0$ , balanced class weights, max 1,000 iterations) is trained on the TF-IDF features. The full sklearn Pipeline bundles the vectoriser and classifier so that serialisation and inference always apply the exact same transformation.

### 3.4 Multilingual Support (translator.py)

At inference, the user’s message language is detected using the langdetect library. If the language is Arabic or French, the text is translated to English using deep-translator (Google Translate, no API key required) before the NLP pipeline runs. Responses can optionally be translated back to the user’s chosen language via the UI language selector.

### 3.5 Case-Based Reasoning Layer (cbr\_engine.py)

A CBR engine adds a retrieval step on top of the ML classifier, following the classic RETRIEVE → REUSE → REVISE → RETAIN cycle:

- RETRIEVE: a separate TF-IDF index (15,000 features, 1–2-grams) is built over ~23,500 curated cases stored in SQLite. Cosine similarity finds the closest matching past case.
- REUSE: the therapeutic response associated with the detected emotion is retrieved from a curated template library (responses.py).
- REVISE: for negative emotions (sadness, anger, hate, empty, anxiety), a cheerful prefix is prepended by the sentiment module to maintain an uplifting tone.
- RETAIN: when a user gives a thumbs-up (👍), the interaction is saved to the SQLite database and the TF-IDF index is rebuilt immediately, enabling online learning from user feedback.

### 3.6 Voice Input (voice/speech\_input.py)

An optional speech recognition module using the SpeechRecognition library and sounddevice allows users to speak their messages instead of typing. The recognised text is then passed through the standard pipeline.

### 3.7 Streamlit Web Interface (app/streamlit\_app.py)

The front-end is built with Streamlit and offers: free-text or voice input, a language selector (EN / AR / FR), a live debug panel showing each pipeline step (detected language, translated text, cleaned tokens, predicted emotion with confidence score), chat history, and a thumbs-up / thumbs-down feedback mechanism.

## 4. Model Used

The production emotion classifier is a scikit-learn Pipeline consisting of two stages:

| Component                 | Configuration                                                                                              |
|---------------------------|------------------------------------------------------------------------------------------------------------|
| <b>Vectoriser</b>         | TfidfVectorizer — max_features=20,000, ngram_range=(1,3), sublinear_tf=True, min_df=3, max_df=0.95         |
| <b>Classifier</b>         | LogisticRegression — C=1.0, multi_class='ovr', class_weight='balanced', max_iter=1,000                     |
| <b>Serialisation</b>      | sklearn Pipeline saved as emotion_classifier.pkl (pickle)                                                  |
| <b>CBR Vectoriser</b>     | TfidfVectorizer — max_features=15,000, ngram_range=(1,2), sublinear_tf=True (separate index for retrieval) |
| <b>Training data size</b> | 39,653 samples across 11 emotion classes                                                                   |
| <b>No. of features</b>    | Up to 20,000 TF-IDF features with negation-tagged vocabulary                                               |

No deep learning or external model APIs are used. The entire system runs locally with no internet requirement after initial dependency installation, ensuring full user privacy.

## 5. Results

### 5.1 Overall Performance

The model achieves 83.70% test accuracy across 21,253 test samples — a strong result for an 11-class emotion classification task on noisy, informal Twitter text.

### 5.2 Per-Emotion Metrics

| Emotion    | Precision | Recall | F1-Score | Support |
|------------|-----------|--------|----------|---------|
| anger      | 0.8692    | 0.8574 | 0.8633   | 1,999   |
| empty      | 0.9570    | 0.8604 | 0.9061   | 1,268   |
| enthusiasm | 0.9635    | 0.9230 | 0.9428   | 2,000   |
| fun        | 0.8920    | 0.8550 | 0.8731   | 2,000   |
| happiness  | 0.7446    | 0.7282 | 0.7363   | 1,998   |
| hate       | 0.9399    | 0.9240 | 0.9319   | 1,999   |
| love       | 0.9075    | 0.9125 | 0.9100   | 1,999   |
| neutral    | 0.7184    | 0.8577 | 0.7819   | 1,996   |

| Emotion                 | Precision     | Recall | F1-Score | Support |
|-------------------------|---------------|--------|----------|---------|
| relief                  | 0.9259        | 0.9120 | 0.9189   | 2,000   |
| sadness                 | 0.6565        | 0.6585 | 0.6575   | 1,997   |
| surprise                | 0.7168        | 0.7261 | 0.7214   | 1,997   |
| <b>Overall Accuracy</b> | <b>83.70%</b> |        |          |         |

### 5.3 Key Observations

- Best-performing classes: enthusiasm (F1 = 0.943), hate (0.932), and relief (0.919) achieve near-human consistency, likely because these emotions carry distinctive lexical patterns.
- Most challenging classes: sadness (F1 = 0.658) and surprise (0.721) are harder to distinguish, as their vocabulary overlaps with neutral and other negative emotions.
- Negation handling: the negation-tagging pre-processor successfully resolves the root cause of mis-classification of phrases such as “I am not happy” (previously misclassified as happiness).
- Multilingual: Arabic and French inputs are correctly routed through translation before classification with no degradation in the detected emotion for direct translations.

## 6. Challenges

---

### 6.1 Negation Misclassification (Resolved)

The most significant bug encountered during development was that phrases such as “I am not happy” were predicted as happiness rather than sadness. Root-cause analysis identified two compounding issues: (1) the word “not” was in the stop-word list and was silently deleted before training, and (2) a train/inference pipeline mismatch meant that even keeping “not” at inference time had no effect because the trained model had never seen it as a feature. This was resolved by introducing the negation-tagging system described in Section 3.1.

### 6.2 Class Imbalance

The raw dataset exhibits extreme class imbalance (up to 77.8:1 across some emotion pairs). This was partially addressed through balanced class weighting in the Logistic Regression classifier, but classes such as sadness and surprise remain comparatively weak.

### 6.3 Informal Language and Twitter Noise

Tweets contain extensive abbreviations, emoji, hashtags, @mentions, and creative spelling that reduce feature quality. The NLP pipeline includes noise removal (Twitter-specific regex patterns), but some informal constructs still fall outside the model’s vocabulary.

### 6.4 Multilingual Quality

Machine translation via Google Translate introduces minor errors for idiomatic or dialect-specific Arabic expressions. These errors propagate into the emotion classification stage. Arabic morphological richness (root-based conjugation) also means that the rule-based lemmatiser is less effective for Arabic input that bypasses translation.

### 6.5 Emotion Boundary Ambiguity

Several emotions in the dataset share overlapping vocabulary (e.g., sadness vs. empty vs. neutral). Human annotators on Twitter data also show low inter-annotator agreement for subtle emotions, which introduces irreducible label noise.

### 6.6 Voice Recognition Constraints

The speech recognition module depends on an active microphone and network access for the default Google Speech API backend. In offline environments, a fallback to a local engine (e.g., Vosk) would be required.

## 7. Future Improvements

---

### 7.1 Model Enhancements

- Fine-tuned transformer model: replacing Logistic Regression with a fine-tuned BERT or AraBERT model would provide context-aware representations that naturally handle negation, sarcasm, and complex sentence structure without handcrafted rules.
- Ensemble classifier: combining the current TF-IDF / Logistic Regression model with a transformer-based model could improve accuracy while keeping the lightweight model as a fast fallback.
- Over-sampling / data augmentation: applying SMOTE or back-translation augmentation on under-represented emotion classes (sadness, surprise) to reduce the impact of class imbalance.

### 7.2 NLP Pipeline

- Arabic-native NLP: integrating a dedicated Arabic NLP library (e.g., CAMEL Tools or Farasa) for proper Arabic tokenisation, diacritisation, and morphological analysis rather than relying solely on translation.
- Sarcasm and irony detection: adding a dedicated sarcasm classifier as a pre-processing gate would prevent sentences like “Oh great, another Monday” from being classified as enthusiasm.
- Expanded contraction and slang dictionary: covering internet slang (lol, smh, rn) and regional Arabic dialect contractions.

### 7.3 System Features

- User personalisation: storing per-user emotion history to track mood trends over time and adjust response warmth accordingly.
- Crisis detection: adding a rule-based safety layer that detects keywords associated with self-harm or crisis situations and routes the user to professional resources.
- Richer response generation: replacing static template responses with a fine-tuned generative model (e.g., a small GPT-2 variant) to produce more varied and contextually relevant therapeutic replies.
- Expanded language support: adding Spanish, German, and French-native NLP (beyond translation) to serve a broader user base.

### 7.4 Deployment & Infrastructure

- Docker containerisation: packaging the application with a reproducible Docker image to simplify deployment across operating systems.
- Streamlit Cloud / HuggingFace Spaces: one-click public deployment for broader accessibility.

- Continuous learning pipeline: automating the RETAIN step to periodically retrain the classifier on accumulated thumbs-up interactions, enabling the model to improve with usage.
- Offline voice recognition: integrating a local speech engine (Vosk or Whisper) to eliminate the microphone's dependency on an internet connection.

---

**Cheerful Bot v2.0** — All processing is local. No user data leaves the machine. No external APIs required at runtime.

---